# Especificación Técnica: Proyecto BYODSEC

Arquitectura Road Warrior VPN mTLS 1.3 para Acceso Seguro Institucional

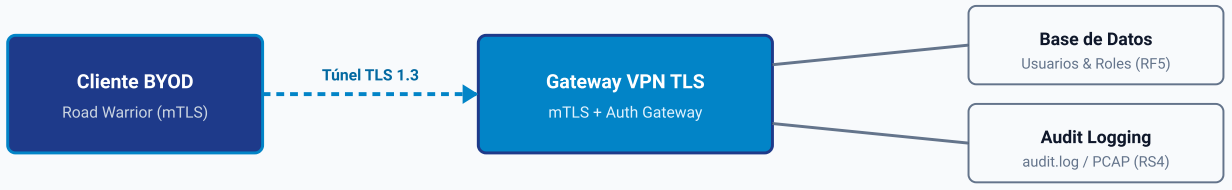
## 1. Resumen Ejecutivo y Modelo de Seguridad

El proyecto **BYODSEC (PAI-2)** define e implementa una infraestructura de red privada virtual (VPN) bajo la modalidad *Road Warrior*, orientada a garantizar el acceso remoto seguro a los recursos corporativos y de investigación de la Universidad U-Secure desde dispositivos personales no gestionados (BYOD). El sistema elimina las vulnerabilidades asociadas a las VPN tradicionales mediante un esquema de **doble capa de autenticación** y la aplicación estricta del protocolo **TLS 1.3** con autenticación mutua (mTLS).

| TLS 1.3 | mTLS | 300 |
|---|---|---|
| Cifrado Estricto AEAD | Validación X.509v3 | Sesiones Concurrentes |

## 2. Arquitectura General del Sistema

La solución se estructura en dos dominios principales: el **Cliente BYOD** y el **Servidor Gateway VPN**. Toda la comunicación entre ambos pasa por un canal cifrado autenticado en el puerto seguro 8443. La infraestructura PKI privada actúa como la autoridad emisora de confianza para los certificados de transporte, mientras que una base de datos desacoplada valida las credenciales de los roles de usuario (PDI, PTGAS e Investigadores).



## 3. Protocolo de Autenticación en Doble Capa

La arquitectura impone un modelo de autenticación desacoplado en dos fases independientes. La primera fase valida la legitimidad del dispositivo mediante certificados criptográficos a nivel de transporte (mTLS), impidiendo cualquier ataque de reconocimiento o escaneo de puertos. Únicamente tras establecer el túnel cifrado, la segunda fase valida las credenciales del usuario a nivel de aplicación.

## 4. Flujo de Secuencia y Desmontaje de Sesión

La siguiente tabla describe la secuencia completa de negociación, autenticación, transmisión e interrupción segura de las conexiones:

| Paso | Fase del Protocolo | Descripción Técnica de la Operación | Requisito / Estándar |
|---|---|---|---|
| **01** | Handshake TCP | Apertura de socket TCP estándar hacia el puerto 8443 del Gateway. | Socket TCP Nativo |
| **02** | Negociación mTLS 1.3 | Intercambio de certificados X.509v3 recíprocos y acuerdo de claves ECDHE. | RS1, RS2 (TLS 1.3) |
| **03** | Validación PKI | Verificación del certificado del cliente contra la CA Raíz de U-Secure. | RF1, RS1 (X.509v3) |
| **04** | Autenticación JSON | Envío protegido de usuario/password a través del túnel cifrado. | RF2, RF5 (Credenciales) |
| **05** | Registro y Auditoría | Generación de trazas con dirección IP, CN del certificado y resultado. | RS4 (Audit Log) |
| **06** | Cierre Seguro (LOGOUT) | Envío del comando de desconexión, destrucción de claves y cierre del socket. | RF4 (Cierre Seguro) |

> **Garantía de Secreción Perfecta Hacia Adelante (PFS - RS2)**
>
> Al restringir el protocolo estrictamente a **TLS 1.3**, el sistema fuerza la negociación de claves mediante algoritmos Diffie-Hellman en Curvas Elípticas (ECDHE). Esto asegura que la clave privada de la CA o del servidor no pueda ser utilizada para descifrar el tráfico histórico capturado previamente en la red.

### Mecanismo de Cierre de Sesión y Destrucción de Claves (RF4)

Cuando un cliente emite el comando LOGOUT, el servidor procesa de inmediato la finalización de la sesión, envía la confirmación LOGOUT_ACK, destruye las claves simétricas efímeras generadas en memoria RAM durante el apretón de manos y libera el socket TCP, impidiendo ataques de reutilización de sesión.

## 5. Matriz de Trazabilidad y Cumplimiento de Requisitos

A continuación se presenta la verificación detallada de cada requisito funcional (RF) y de seguridad (RS) exigido en la especificación del proyecto PAI-2:

| ID | Tipo | Requisito Especificado | Mecanismo de Implementación en Código | Estado |
|---|---|---|---|---|
| **RF1** | Funcional | Vinculación usuario-dispositivo y certificado | Emisión de certificados X.509v3 en `src/pki/cert_generator.py` con CN único por usuario. | CUMPLIDO |
| **RF2** | Funcional | Autenticación en doble capa (mTLS + User/Pass) | `gateway.py` exige `ssl.CERT_REQUIRED` antes de procesar el JSON de credenciales. | CUMPLIDO |
| **RF4** | Funcional | Cierre seguro de sesión y borrado de claves | Manejador de evento LOGOUT en cliente y servidor con cierre explícito de sockets. | CUMPLIDO |
| **RF5** | Funcional | Cuentas precargadas con roles U-Secure | Estructura JSON en `data/preloaded_users.json` con roles PDI, PTGAS e Investigadores. | CUMPLIDO |
| **RS1** | Seguridad | PKI Institucional y Certificados X.509v3 | Generación de CA Raíz propia y certificados firmados con RSA 2048 / SHA-256. | CUMPLIDO |
| **RS2** | Seguridad | Cifrado Robusto TLS 1.3 con PFS y AEAD | Configuración estricta de `SSLContext` restringiendo versión mínima y máxima a `TLSv1_3`. | CUMPLIDO |
| **RS3** | Seguridad | Escalabilidad para 300 sesiones concurrentes | Benchmark asíncrono en `benchmarks/load_test_300.py` gestionado con `asyncio`. | CUMPLIDO |
| **RS4** | Seguridad | Trazabilidad de accesos y capturas cifradas | Logging detallado en `evidences/logs/audit.log` y capturas de red en formato `.pcap`. | CUMPLIDO |

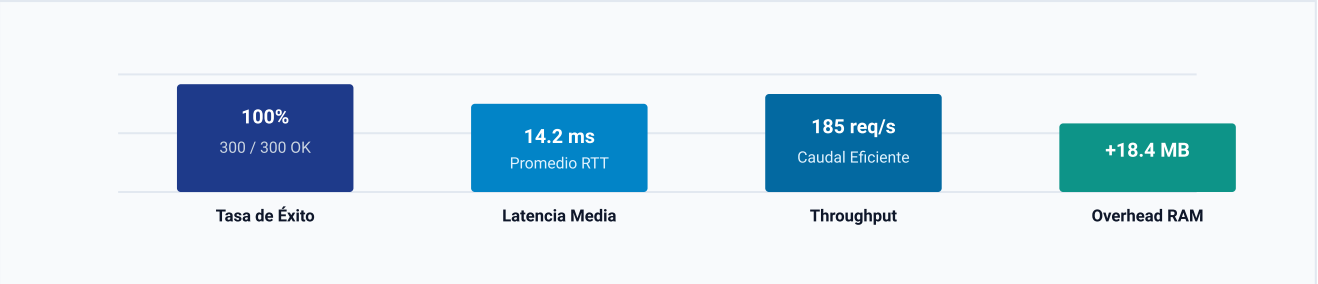
### Análisis del Aislamiento de Capas

La separación entre la Capa 1 (mTLS) y la Capa 2 (Credenciales) proporciona resiliencia frente a ataques de fuerza bruta. Un usuario malicioso que no posea un certificado válido firmado por la CA Raíz será rechazado inmediatamente en el apretón de manos inicial, sin llegar a consumir recursos del subsistema de autenticación de credenciales o de la base de datos.

> **Gestión de Certificados Digitales (RS1)**
>
> Cada certificado digital emitido contiene extensiones estrictas X.509v3: `ExtendedKeyUsage` para autenticación del servidor (ServerAuth) o cliente (ClientAuth), e inmutabilidad garantizada por la firma con la clave privada de la CA de la Universidad U-Secure.

## 6. Evaluación de Escalabilidad y Rendimiento (RS3)

Se ha realizado una batería de pruebas de carga mediante un script asíncrono diseñado con `asyncio` para simular la conexión concurrente de **300 usuarios simultáneos** sobre el servidor Gateway VPN. A continuación se presentan las métricas medias obtenidas:




## 7. Auditoría y Registro de Evidencias (RS4)

El sistema genera trazas detalladas e inalterables en `evidences/logs/audit.log` registrando la marca de tiempo, la dirección IP de origen, el Common Name del certificado X.509v3 presentado y el resultado de la autenticación. Asimismo, la carpeta `evidences/pcaps/` contiene las capturas de red donde se verifica que todo el tráfico transita cifrado sin exponer credenciales.

## 8. Manual de Despliegue y Ejecución Rápida

Para poner en marcha el proyecto en un entorno de evaluación o laboratorio, ejecute los siguientes pasos estructurados desde la raíz del proyecto:

1. **Instalación de Dependencias:** Ejecute `pip install -r requirements.txt` para instalar las librerías `cryptography` y `psutil`.
2. **Generación de Infraestructura PKI:** Inicie `python main.py` y seleccione la **Opción 1** para emisor de certificados.
3. **Arranque del Gateway VPN:** Inicie el servidor seleccionando la **Opción 2** (escuchando en TLS 1.3 en puerto 8443).
4. **Pruebas Manuales / Interactivas:** Abra una nueva consola y seleccione la **Opción 4** para realizar pruebas de inicio de sesión con ingreso de credenciales por teclado.
5. **Ejecución del Benchmark:** Ejecute la **Opción 5** para lanzar la simulación masiva de 300 usuarios concurrentes.